Node.js — Day 27

Backend Flow, Service Modules & Business Logic

■ Today’s Goal

Bring together the Node.js concepts learned so far and understand how a small backend can be organized into focused modules. The focus was the flow between **app.js**, **userService.js**, **user.js**, and **validation.js**.

1. Backend Flow

```
process.argv
    ↓
app.js
    ↓
validation.js
    ↓
userService.js
    ↓
user.js
    ↓
Result
    ↓
app.js prints output
```

app.js acts as the coordinator: it receives input, asks validation to check it, calls the user service, and prints the result.

2. Service Modules

A **service** is a module that handles a particular area of application operations or logic. In a larger backend, you might have user, product, and order services.

In our project, **userService.js** sits between **app.js** and **user.js**, creating a cleaner separation of responsibilities.

3. userService.js

```
const createUser = require("./user");

function registerUser(name, age) {
    const user = createUser(name, age);

    return user;
}

module.exports = registerUser;
```

The service imports **createUser**, calls it through **registerUser()**, and returns the resulting user object.

4. Connecting the Service to app.js

```
const registerUser = require("./userService");

const user = registerUser(name, age);
```

Instead of directly calling **createUser()**, **app.js** now communicates with the user service.

```
app.js
  ↓
registerUser()
  ↓
userService.js
  ↓
createUser()
  ↓
user.js
  ↓
User object
  ↓
app.js prints result
```

5. Business Logic Practice

Business logic represents application rules and decisions. We added a user status rule based on age:

```
Age < 18       → "Minor"
Age 18–59      → "Adult"
Age 60+        → "Senior"
```

This rule belongs with user-related business logic rather than being placed directly inside **app.js**.

6. Testing Multiple Inputs

```
node app.js Bhavana 16
→ Status: Minor

node app.js Bhavana 26
→ Status: Adult

node app.js Ravi 65
→ Status: Senior
```

All three test cases were executed successfully.

7. Responsibilities of Each Module

Module	Main Responsibility
app.js	Coordinates the application flow and prints output.
validation.js	Checks whether required input is valid.
userService.js	Handles the user-related operation and connects app.js to user logic.
user.js	Contains user-related business logic and creates the user object.

8. Why Separate Modules?

Separating responsibilities makes the backend easier to **read, organize, maintain, test, and extend**. Instead of placing everything in one large file, related responsibilities stay together.

Mental model:
app.js = Coordinator
validation.js = Input checker
userService.js = User operation/service layer
user.js = User business logic

9. Final Assessment

Result: **3/3** ■

Q1 — userService.js organizes user-related operations separately.
Q2 — The age/status business rule belongs in user.js.
Q3 — Separation makes the code easier to read, organize, maintain, and extend.

■ Day 27 — COMPLETE

You explained the backend flow, created and connected a service module, implemented business rules, tested multiple inputs, and passed the final assessment.

■ Next: Small Node.js Practice Project

The next session will bring the Node.js foundation together in a small practical project before final revision and assessment.